

نام شرکت کنندگان	مشخصات	تصویر
آناهیتا مرادی	کارشناسی ارشد معماری_معماری	
الناز محمدبیگی	کارشناسی ارشد معماری_معماری	
درنا عبداللهی فرد	کارشناسی ارشد معماری_پایدار	

Morphotect



پروژه بوتکمپ

طراحی
"مدل‌های دیجیتال انسان محور
مبتنی بر ادراکات محیطی در معماری و شهرسازی"

تاریخ ارائه: ۱۴۰۳/۶/۲۳

۱.۲ طرح موضوع مورد مطالعه



در این مطالعه، به بررسی نقش تعیین ورودی‌های باکیفیت از نظر ادراک بصری در طراحی بلوک‌های مجتمع مسکونی اکباتان به عنوان یکی از شاخص‌ترین مجتمع‌های مسکونی ایران پرداختیم.

این مجموعه به دلیل وسعت زیاد و همچنین احجام صلب ساختمانی بزرگ، چالش‌هایی از جمله عدم خوانایی در مسیریابی الاخصوص در بحث پیدا کردن ورودی به بلوک‌ها ایجاد می‌نماید.

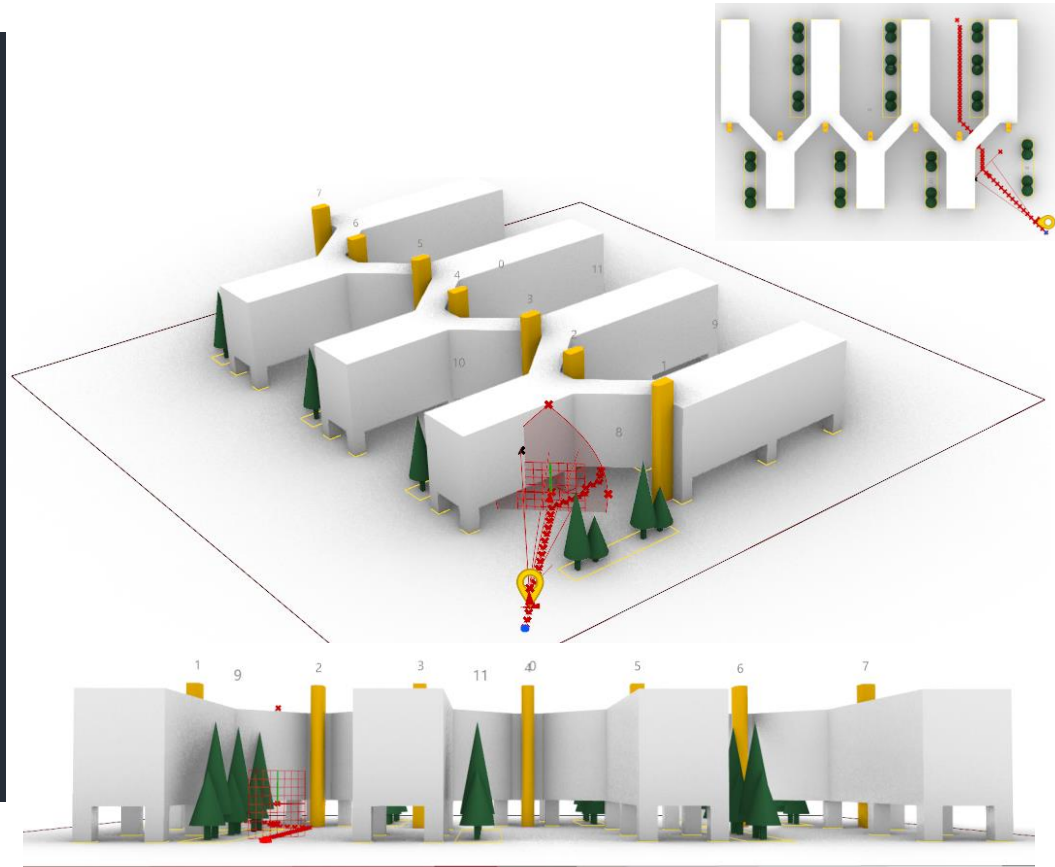
از طریق ادراک بصری هرچقدر فضا بیشتر دیده شود حس خوانایی فضا بیشتر خواهد شد.

۱.۳ پیشنهاد گروه برای روش حل مسئله

برای حل مسئله مذکور، الگوریتمی با کمک VISION TRACKING و A* طراحی کردیم تا با تحلیل داده‌های خروجی از آن به میزان ادراک بصری هر ایجنت در محیط مجتمع پی ببریم.

حرکت ایجنت با در نظر گرفتن VISION TRACKING می‌تواند دیتاهای مسیر را جمع‌آوری کند و با در نظر گرفتن میزان خوانایی مسیر به ایجنت در مسیریابی کمک کند.

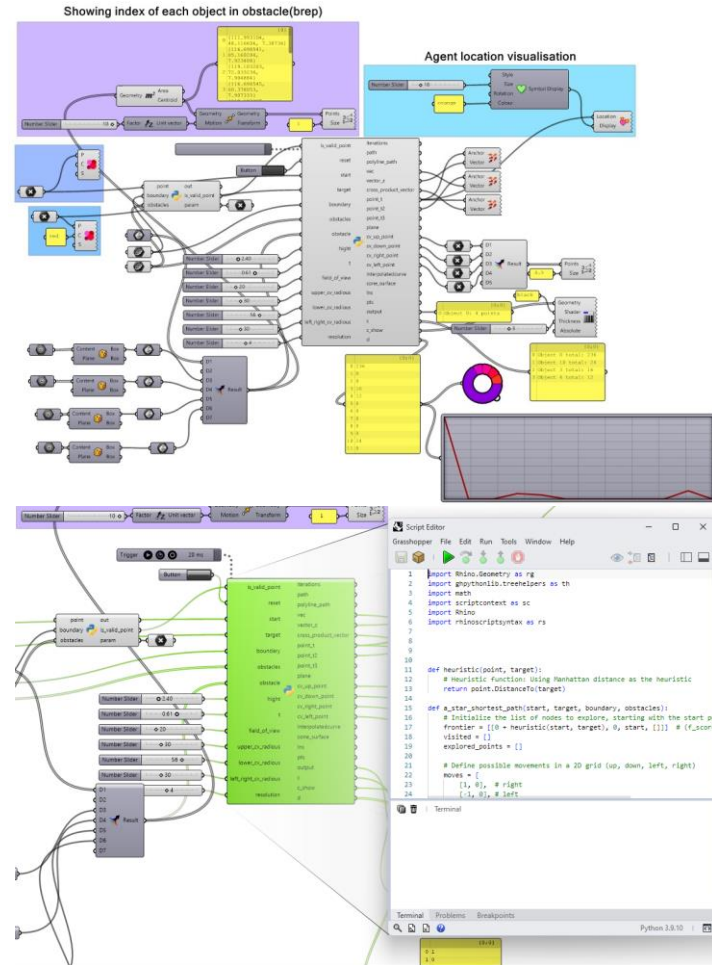
در مجتمع با ساختمان‌های مدولار، از آنجایی که نمای ساختمان‌ها به صورت یکپارچه و وسیع دیده می‌شوند و در طول مسیر به هم شبیه‌اند، باید نشانه‌های بصری متمایز برای نقشه‌های شناختی افراد تعریف کرد.



۱.۴ منطق حل الگوریتم و توضیح ساختار کد (سودو کد)

در این بررسی از کدهای **VISION TRACKING & A*** استفاده شده است.

- **A*** ضمن شناخت موانع، کوتاه‌ترین مسیر تا هدف مشخص شده را بررسی می‌نماید.
- خروجی **A*** پس از در نظر گرفتن مرز محدوده مشخص شده و همچنین موانع موجود در بستر طراحی به صورت نقطه تعریف می‌شود. سپس این نقاط به یکدیگر متصل می‌شوند.
- در مرحله بعد **AGENT** با لحاظ ارتفاع مناسب از سطح زمین به عنوان قد ناظر، روی خط تعریف می‌شود.
- سپس با محاسبه حاصل ضرب خارجی بردار مماس بر مسیر و همچنین بردار قد ناظر، صفحه کلی دید ناظر با فاصله مشخص تعریف شد.
- از آنجایی که دید بالا و پایین انسان مشابه هم نیستند یک متغیر به عنوان زاویه دید بالا، پایین، چپ و راست به عنوان ورودی‌هایی که اسکریپت پایتون متصل می‌شوند تعریف شد.
- در این شرایط در نظر گرفتیم که ناظر دید محدودی از صفحه دارد. بنابراین یک بردار با زاویه ۷۰ درجه از چشم ناظر روی صفحه تعریف شده، ایجاد می‌شود.
- در نهایت با بردارهای ایجاد شده، ۴ نقطه با زاویه دید ناظر روی صفحه مشخص کردیم. با اتصال این نقاط به دیگر صفحه‌ای ایجاد می‌گردد و به عنوان محدوده دید ناظر تعریف شد.
- سپس برای مشخص شدن نقاط تلاقی حاصل از خطوطی که از چشم ما به صفحه می‌رسند در تلاقی با آبجکت‌ها.
- با متغیر **RESOLUTION** هر خطی که از سمت چشم ناظر به صفحه تعریف شده دید آن ساطع می‌شود باید در صفحه مذکور دیده شود.
- در نتیجه یک خط از دید چشم ما به صفحه متصل می‌شود.
- در مرحله بعد برای تعیین نقاط تلاقی، از دستورات **(for)** و **(if)** شرط استفاده شد تا تلاقی هر خط ساطع شده از لیست خطوط، با هر آبجکت از لیست موانع بدست آید.



ساختر کد (سودو کد)

```

1 import math
2 import sys
3 import math
4 import sys
5 import sys
6 import sys
7 def heuristic(point, target):
8     # Heuristic function using Manhattan distance as the heuristic
9     return abs(point[0]-target[0]) + abs(point[1]-target[1])
10
11 def a_star_shortest_path(start, target, boundary, obstacles):
12     # Initialize the list of nodes to explore, starting with the start point
13     frontier = [(0, heuristic(start, target), 0, start, [])] # (f_score, g_score, position, path)
14     visited = []
15     explored_points = []
16     # Define possible movements in a 2D grid (up, down, left, right)
17     moves = [
18         (-1, 0), # left
19         (1, 0), # right
20         (0, -1), # up
21         (0, 1), # down
22         (-1, -1), # left-down
23         (-1, 1), # left-up
24         (1, -1), # right-down
25         (1, 1), # right-up
26     ]
27     while frontier:
28         counter = 1
29         # Sort frontier to always expand the node with the lowest f_score first
30         frontier.sort()
31         current_f_score, current_g_score, current_position, path = frontier.pop(0)
32
33         # Check if we have reached the target
34         if current_position == target:
35             return path + [target], explored_points, counter
36
37         # Mark the current position as visited
38         if current_position not in visited:
39             visited.append(current_position)
40
41         # Explore all possible moves
42         new_points = []
43         for move in moves:
44             new_position = (current_position[0] + move[0], current_position[1] + move[1])
45             new_f_score = current_f_score + current_position.DistanceTo(new_position) # Assuming uniform cost for each move
46             new_g_score = heuristic(new_position, target)
47
48             # Check if the next position is within an obstacle
49             is_valid = 1
50             is_valid = is_valid and not (new_position in obstacles)
51             is_valid = is_valid and not (new_position in boundary)
52
53             # If it's not in an obstacle and hasn't been visited, add it to the list with its f_score
54             if is_valid and new_position not in visited:
55                 frontier.append((new_f_score, new_g_score, new_position, path + [current_position]))
56                 new_points.append(new_position)
57             explored_points.append(new_points)
58
59         return [None, explored_points, counter] # Return None if no path is found
60
61 path, explored_points, iterations = a_star_shortest_path(start, target, boundary, obstacles)
62
63 explored_points = th.list_to_tree(explored_points)
64
65 # Constructed path
66 poly_line_path = rg.PolyLine(path)
67 poly_line_path.Domain = rg.Interval(0,1)
68 # Point to poly_line_path.PointSet()
69
70 # Moving agent
71 import Rhino.Geometry as r
72 import ghpythonlib.treehoppers as th
73
74 if "para" not in globals() or reset:
75     para = 0
76     t_para = 0.05
77
78 if "intersected_pts" not in globals() or reset:
79     intersected_pts = []
80
81 vec2 = poly_line_path.TangentAt(para)
82 para = t_para
83 if para > 1:
84     para = 0
85 elif para < 0:
86     para = 1
87     t_para = 0.05
88
89 # tangent_vec = path.TangentAt(para)
90 point_t = poly_line_path.PointAt(para)
91 pt1 = point_t + vec2 * (t)
92
93 # Agent eye height
94 vector_vrg = Vector3d(0,0,height)
95 point_t1point_t = vector_t

```

```

96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241

```

```

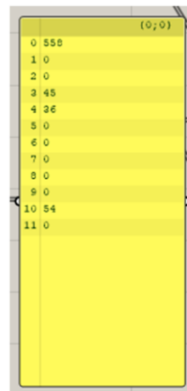
182 if "all_intersected_pts_show" not in globals() or reset:
183     all_intersected_pts_show = [] # Global list to store all intersected points show
184
185 # Dictionary to store points for the closest object
186 closest_pts_dict = {}
187 pts = []
188 intersected_pts = []
189
190 for line in lns:
191     mindis = float('inf')
192     closestpoint = None
193     closest_index = None # To track the index of the closest object
194
195     # Iterate over each obstacle to find the closest one
196     for idx, obj in enumerate(obstacle):
197         intersected_pts = rg.Intersect.IntersectCurveRep(line, obj, 0.01)[2]
198
199         for i in intersected_pts:
200             distance = point_t1.DistanceTo(i)
201             if distance < mindis:
202                 mindis = distance
203                 closestpoint = i
204                 closest_index = idx # Update the index of the closest object
205
206     # If a closest point was found, add it to the dictionary for the closest object
207     if closestpoint is not None and closest_index is not None:
208         if closest_index not in closest_pts_dict:
209             closest_pts_dict[closest_index] = []
210         closest_pts_dict[closest_index].append(closestpoint)
211         pts.append(closestpoint)
212
213 all_intersected_pts_show.extend(pts)
214
215 for idx, points in closest_pts_dict.items():
216     if idx not in all_intersected_pts:
217         all_intersected_pts[idx] = []
218         all_intersected_pts[idx].extend(points)
219
220 # Prepare output for the number of points for the closest object
221 a = []
222 if closest_index is not None:
223     a.append(f"Closest Object {closest_index}: {len(closest_pts_dict[closest_index])} points")
224
225
226 # Output the number of intersection points for each object by their index
227 b = {idx: len(points) for idx, points in closest_pts_dict.items()}
228 output = [f"Object {idx}: {count} points" for idx, count in b.items()]
229
230 c = [f"Object {idx} total: {len(points)}" for idx, points in all_intersected_pts.items()]
231
232 c_show = all_intersected_pts_show
233
234 # Initialize output panel d with indexes and zero intersections
235 d = [f"0" for idx in range(len(obstacle))]
236
237 # Update d with the number of intersections for each index
238 for idx, points in all_intersected_pts.items():
239     d[idx] = f"{len(points)}"
240
241

```

۱.۵ نمایش خروجی ها و دلیل انتخاب این شیوه از نمایش داده ها

برای این کار به عنوان بررسی یک نمونه ساخته شده و ساده شده، یک بلوک از فاز ۱ شهرک اکباتان انتخاب شد. بلوک، درخت ها و باکس های پله به عنوان موانع در نظر گرفته شد.

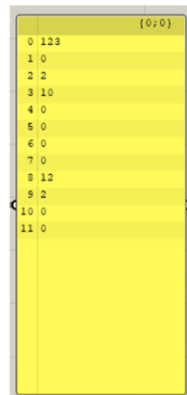
باکس شماره ۳ به عنوان مکان هدف انتخاب شد. برای ادامه ی کار، دو نقطه شروع متفاوت به ایجنت داده شد تا حرکت در میان موانع، در کنار دید او در ۲ مسیر مورد مقایسه قرار گیرد. خوانایی باکس شماره ۳ با انتخاب مسیر کوتاه توسط ایجنت و دیده شدن زودتر باکس شماره ۳ ارتباط پیدا می کند.



start: top right entrance

target: object index ۳

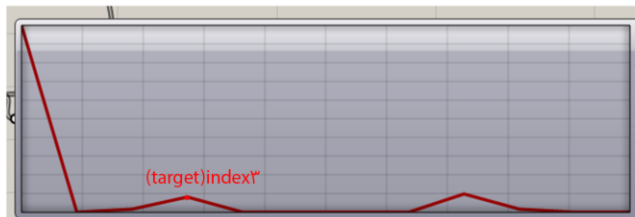
first seen: step ۴



start: top left entrance

target: object index ۳

first seen: step V



۱.۶ کاربردهای الگوریتم در معماری و شهرسازی

کاربرد در معماری

کتابخانه استپلتون، نیویورک^۱



مردم نماهای خالی را نادیده میگیرند.
کمک به معماران برای طراحی نماهای جذابتر و بصری تر.

بهینه سازی مراکز خرید^۲



قرار دادن مؤثر تابلوها و نمایشگرها.
جلب توجه مشتری و بهبود مسیریابی.

کاربرد در طراحی شهری

رفتار عابرین پیاده^۳



بررسی حرکت افراد در تقاطعها و فضاهای عمومی.
کمک به طراحی مسیرهای عابر پیاده ایمن تر و بصری تر.

نظریه "چشم انداز سریال"
گوردون کالن^۴



تقویت نظریه با ردیابی چشم
ردیابی مسیر نگاه افراد در فضاهای شهری.
جمع آوری داده های دقیق درباره تعامل و واکنش. کمک به طراحی فضاهای بصری جذابتر و هماهنگ تر.

۱.۷ محدودیتهای پروژه (مهم ترین چالش های گروه برای حل مسئله)

- در صورتی که بخواهیم از این الگوریتم در مقیاس وسیع استفاده کنیم، میتوانیم دریابیم که در مسیرهای حرکتی انتخابی آدم ها (که مسیر کوتاه را انتخاب میکنند)، چه موانعی (آجکت ها، نماها و ...) برای دیدن وجود دارد، و با تحلیل میزان دید ایجنت نسبت به این موارد، میتوانیم مکان بهتری برای ورودی در نظر بگیریم.

۱.۸ پیشنهادات آتی برای پژوهشگران

- در انجام این پروژه حرکت سر ایجنت در نظر گرفته نشده است، بنابراین پیشنهاد می شود در پژوهش های بعدی این مهم در نظر گرفته شود.
- ضمناً در ارتباط مجتمع مسکونی اکباتان به عنوان یکی از مجتمع های مهم در شهر تهران می توان به بحث هایی پیرامون تاثیر طراحی انسان محور بر روی مواردی از جمله امنیت و تعاملات اجتماعی ساکنین پرداخت.

منابع نمونه موردی

- 1) 1. Sussman, A., & Ward, J. M. (2017, December 6). Here's what you can learn about architecture from tracking people's eye movements. ArchDaily. Retrieved September 13, 2024, from <https://www.archdaily.com/884945/heres-what-you-can-learn-about-architecture-from-tracking-peoples-eye-movements2>.
- 2) Harwood, T., & Jones, M. (2014). Mobile eye-tracking in retail research. In M. Horsley, M. Eliot, B. Knight, & R. Reilly (Eds.), Current trends in eye tracking research (pp. 237-251). Springer, Cham. https://doi.org/10.1007/978-3-319-02868-2_143.
- 3) Gruden, C., Ištoka Otković, I., & Šraml, M. (2022). An eye-tracking study on the effect of different signalized intersection typologies on pedestrian performance. Sustainability, 14(4), 2112. <https://doi.org/10.3390/su140421124>.
- 4) Urban CGI. (n.d.). Gordon Cullen: Serial vision in urban design. Retrieved from <https://www.urbandgi.com/articles/gordon-cullen-serial-vision-in-urban-design/>